

Final Project Proposal

Prediction to Prescription: Building an AI Agent

Team: [Team Number] **Project Title:** Uniform Ordering Optimization for a Youth Soccer League

Date: [DUE DATE]

Name	Email	Program	Primary Role
<i>Member 1</i>	<i>email@purdue.edu</i>	<i>Program</i>	<i>Problem Formulation Module</i>
<i>Member 2</i>	<i>email@purdue.edu</i>	<i>Program</i>	<i>Data Collection & Preparation</i>
<i>Member 3</i>	<i>email@purdue.edu</i>	<i>Program</i>	<i>Prediction Layer</i>
<i>Member 4</i>	<i>email@purdue.edu</i>	<i>Program</i>	<i>Optimization Engine</i>
<i>Member 5</i>	<i>email@purdue.edu</i>	<i>Program</i>	<i>Explanation & Validation</i>

Section 1 — Problem Statement and Business Context

1.1 Business Context

CFA (Club Futbol Academy) is a youth soccer league based in Orange County, California, operating four seasonal programs — fall, winter, spring, and a year-round competitive (CFA) division. Each season, the league’s president must decide how many official game uniforms to order across three product categories — jerseys (tops), shorts (bottoms), and socks — in each available size before registration demand is fully realized. The league currently processes approximately 2,000–3,000 uniform orders per season through its Squarespace storefront, serving hundreds of youth players across multiple age groups and gender divisions.

Today, order quantities are determined by the league president using 25 years of operational intuition. This heuristic works reasonably well on average, but produces systematic errors in both directions: overstock on some sizes (tying up capital in inventory that children outgrow by the next season) and stockout on others (triggering rush orders with premium shipping costs, delayed fulfillment, and dissatisfied families).

The problem is compounded by Adidas’s jersey lifecycle. Each jersey model (e.g., Condivo 22, Tiro 25) is produced for approximately two years before being discontinued and replaced. This creates a **two-period decision structure** with escalating costs:

- **Year 1 overage:** Unused jerseys carry to Year 2 at minimal holding cost.
- **Year 1 underage:** Rush orders at the same unit price but with added shipping costs and fulfillment delays.
- **Year 2 overage:** Excess inventory is sold on eBay at a salvage price, incurring a loss.
- **Year 2 underage:** The jersey model has been discontinued by Adidas. Sourcing secondhand stock requires paying significant premiums, or the sale is lost entirely — resulting in lost revenue and unhappy parents.

1.2 Problem Class

This is a **multi-period, multi-product newsvendor ordering problem**. Each season, the league decides how many units of each size to order for jerseys, shorts, and socks before demand is fully observed. The goal is to minimize total procurement cost — including holding costs, rush-order shipping premiums, salvage losses, and stockout penalties — subject to a seasonal budget constraint and supplier minimum order quantities. Demand by size is uncertain and shifts across seasons because children grow, registration counts vary, and the mix of age groups changes. The two-year jersey lifecycle adds a temporal coupling constraint: Year 1 ordering decisions directly affect Year 2 feasibility and cost through carryover inventory.

1.3 Why This Problem Is Hard Without an Agent

Three factors make this problem difficult for a human to solve well without algorithmic support. First, **demand by size is uncertain and non-stationary**: youth players grow between seasons, so last year’s size distribution is a biased estimate of next season’s demand. Second, **the cost structure is highly asymmetric and escalating**: as the course’s Bertsimas–Kallus framework establishes, minimizing prediction error (MSE) is not the same as minimizing decision cost when overage and underage costs differ — and in Year 2 of a jersey lifecycle, the underage cost can be an order of magnitude higher than the overage cost. A fixed ordering heuristic cannot capture this asymmetry. Third, **the problem is combinatorial across sizes**: ordering decisions for each size interact through a shared budget constraint, meaning the optimal order for size YM depends on what is ordered for every other size.

We expect the agent to outperform the current heuristic by learning the cost-asymmetric demand distribution from six years of historical order data and producing size-level ordering recommendations that minimize expected total cost — not just prediction error.

Section 2 — Data Plan

2.1 Primary Dataset

Field	Details
Dataset name	CFA Squarespace Order History
Source / URL	Exported directly from the league’s Squarespace e-commerce backend (proprietary; league president has authorized use)
Approximate size	90,010 rows $\times$ 52 columns; date range August 2017 – May 2026
Key features	Order date, product name, product category (top/bottom/socks), size variant, quantity, unit price, uniform set (fall/winter/spring/CFA), gender/age group
Outcome variable	Demand quantity per size per product category per season
Known limitations	Size field is embedded in a free-text variant string (requires parsing); product names are inconsistent across seasons (requires normalization); no explicit jersey model/year-in-lifecycle flag (must be inferred from product name and order date)

After cleaning, the dataset reduces to **12,209 official game uniform line items** spanning 2020–2026, across 15 normalized size codes, 4 uniform sets, and 3 product categories. Player names have been removed for privacy. The cleaning pipeline is reproducible via `scripts/clean_data.py` in the project repository.

2.2 Additional Datasets (if any)

Registration data (pending): player-level registration records including age, gender, and team assignment. This will enable richer size-demand prediction by connecting demographic features to sizing patterns. The league president has agreed to provide this data.

Adidas jersey lifecycle data: release and discontinuation dates for Adidas teamwear models (Condivo 20, Condivo 22, Tiro 25, Tiro 26), sourced from public retailer databases and Footy Headlines. Used to label each order with its position in the two-year jersey lifecycle.

2.3 Train / Test Split

Chronological split: **train on 2020–2024** (approximately 9,500 line items across 8–10 seasons), **test on 2025–2026** (approximately 2,700 line items across 4 seasons). This preserves temporal ordering and prevents data leakage from future seasons into training. The test set includes seasons from both Year 1 and Year 2 of the current jersey lifecycle, enabling evaluation of the agent’s performance under both cost regimes.

Section 3 — Methods Sketch

3.1 Prediction Layer

Following the Bertsimas–Kallus framework from Lectures 2 and 4, we will compare three prediction approaches with increasing decision-awareness:

1. **PTO-Naive (MSE regression)**: Fit a demand model by minimizing mean squared error. Use the predicted demand as the order quantity. This is the “wrong loss” baseline — it treats over-prediction and under-prediction as equally costly.
2. **PTO-Adjusted (MSE + critical ratio)**: Same MSE regression, but adjust the order using the newsvendor critical ratio quantile: $q^* = \hat{D} + \sigma \cdot \Phi^{-1}(\frac{c_u}{c_u + c_o})$. This partially corrects for cost asymmetry but still trains the demand model on prediction error, not decision cost.
3. **End-to-End Prescriptive (NV loss + Adam)**: Train a linear prescription model directly on newsvendor cost using the Adam optimizer, as implemented in the FreshChoice case notebook. The model output is the *order quantity itself*, not a demand forecast. The gradient of the newsvendor cost flows back through the model parameters via the chain rule:

$$\frac{\partial \mathcal{L}_{NV}}{\partial \beta} = \underbrace{\frac{\partial C}{\partial q}}_{\pm c_o / -c_u} \cdot \underbrace{\frac{\partial q}{\partial \hat{D}}}_{e^{\sigma \Phi^{-1}}} \cdot \underbrace{\frac{\partial \hat{D}}{\partial \beta}}_{\hat{D} \mathbf{x}}$$

This allows the model to develop intentional biases that reduce cost — for example, systematically over-ordering sizes where underage costs dominate (Year 2 of the jersey lifecycle).

The Data Scientist Agent will run all three approaches, compare their performance on the holdout set using actual newsvendor cost (not RMSE), and select the best performer. Features available at decision time include: season, uniform set, year in jersey lifecycle, historical size distribution, and (when available) registration demographics.

3.2 Optimization Engine

The constrained multi-product newsvendor is solved using `scipy.optimize.minimize` with the SLSQP method, following the approach from Lectures 2–3 and the FreshChoice notebook. The objective is the Sample Average Approximation (SAA) of the expected newsvendor cost across all sizes:

$$\min_{\mathbf{q}} \frac{1}{N} \sum_{i=1}^N \sum_{s \in \text{sizes}} C_s(q_s, D_{s,i}) \quad \text{where} \quad C_s(q, D) = c_o^{(s)}(q - D)^+ + c_u^{(s)}(D - q)^+$$

subject to budget, minimum order, and non-negativity constraints. The LLM-based Data Scientist Agent generates the solver code from the `ProblemConfig`, adapting the cost parameters (c_o , c_u) based on the jersey lifecycle year and product category.

Shadow price analysis (per Lecture 3, KKT conditions) will quantify the business cost of each binding constraint — enabling the Reporting Agent to tell the league president, for example, “relaxing the budget by \$500 would reduce expected season cost by \$X.”

3.3 Constraint Handling

Constraints are enforced as hard constraints passed directly to the SLSQP solver:

- **Budget constraint:** Total procurement cost $\sum_s p_s \cdot q_s \leq B$ (seasonal budget).
- **Minimum order:** $q_s \geq q_{\min}$ for each size (supplier minimum).
- **Non-negativity:** $q_s \geq 0$ for all sizes.

When the unconstrained prescriptive model’s orders violate constraints, the harness applies a projection step (as in Lecture 3, Projected SGD) to map the solution back to the feasible set before presenting it to the user.

3.4 Baseline Comparison

The baseline is the league president’s current practice: ordering based on historical averages and 25 years of intuition, without systematic adjustment for cost asymmetry or size-distribution shifts. We approximate this baseline from the data as a **fixed-order policy** — the average order quantity per size across training seasons — following the same “produce manager orders the historical average” baseline used in the FreshChoice case.

The agent’s recommendations will be compared to this baseline on the holdout test set using **total season newsvendor cost** (procurement + rush shipping + salvage loss + stockout penalty), not prediction accuracy.

3.5 Pipeline Diagram

The system follows a three-agent sequential architecture:

User (NL description) $\rightarrow$ Problem Formulator Agent $\rightarrow$ ProblemConfig $\rightarrow$ Data Prep
(Python) $\rightarrow$ Data Scientist Agent

(prediction: PTO vs. prescriptive NV loss | optimization: SLSQP + SAA)

$\rightarrow$ Reporter Agent $\rightarrow$ Dashboard + PDF Report

Each agent saves structured JSON output to a shared run directory. The next agent loads previous outputs — no direct coupling. Every step is logged to a trace for audit and explainability.

Section 4 — Team Roles

Component	Primary Owner	Brief Description of Responsibility
Problem Formulation Module	[Name]	Chat-based NL intake agent; ProblemConfig schema
Data Collection & Preparation	[Name]	Cleaning pipeline, feature engineering, train/test split
Prediction Layer	[Name]	PTO and prescriptive NV-loss models; model comparison
Optimization Engine	[Name]	SLSQP solver, constraint handling, shadow price analysis
Explanation & Validation	[Name]	Reporter agent, dashboard, sensitivity analysis, PDF expo

All team members will collaborate across components. The Data Collection module is largely complete (cleaning pipeline delivered). Role assignments are not binding but ensure clear ownership for progress tracking.

Section 5 — Benchmark and Validation Plan

5.1 Evaluating Your Own Problem

The primary metric is **total season newsvendor cost** on the held-out test seasons (2025–2026):

$$\text{Total Cost} = \sum_{s \in \text{sizes}} [c_o^{(s)} \cdot (q_s - D_s)^+ + c_u^{(s)} \cdot (D_s - q_s)^+]$$

where c_o and c_u vary by product category and jersey lifecycle year. We will report:

- Average season cost reduction (%) vs. the fixed-order baseline.
- Stockout rate and overstock rate per size.
- Percentage improvement from PTO-Adjusted over PTO-Naive, and from End-to-End Prescriptive over PTO-Adjusted — isolating the value of cost-aware training.

A cost reduction of 10–15% over the baseline would be practically meaningful for a league of this size, representing hundreds of dollars per season in reduced waste and rush-order costs.

5.2 Stakeholder Robustness

What is parameterized: The entire problem definition is captured in a **ProblemConfig** JSON schema — decision variables, objective direction, constraints (with parameters), cost structure (c_o , c_u per product and lifecycle year), and data requirements. The Problem Formulator Agent accepts natural language input from the user and generates this config; the user can review and edit any field before the pipeline runs.

How a new problem description is communicated: The instructor (acting as stakeholder) describes the modification in natural language. The Problem Formulator Agent updates the **ProblemConfig** accordingly. Examples of changes the agent can handle without re-engineering:

- A new constraint (e.g., “add a sustainability cap of 500 total units”) → add to `constraints` list, re-run optimizer.
- A shifted objective (e.g., “maximize service level instead of minimizing cost”) → update `objective` field, regenerate solver code.
- Changed costs (e.g., “rush shipping now costs 2× more”) → update `cost_structure`, re-run optimizer + explanation.
- A new decision (e.g., “also decide how many number transfers to pre-order”) → add to `decision_variables`, may trigger solver code regeneration.

Limits of adaptability: The agent handles any change expressible in `ProblemConfig` fields. A fundamentally different problem class (e.g., switching from inventory ordering to staff scheduling) or a new data source not present in the existing dataset would require re-engineering of the data and prediction modules.

References

- [1] Bertsimas, D. & Kallus, N. “From Predictive to Prescriptive Analytics.” *Management Science*, 66(3), 1025–1044, 2020.
- [2] Kingma, D.P. & Ba, J. “Adam: A Method for Stochastic Optimization.” *ICLR*, 2015.
- [3] Jaiswal, P. “From Prediction to Prescription: Why Optimization Projects Fail in Practice.” *MGMT 590-037 Lecture Notes*, Summer 2026.
- [4] Jaiswal, P. “SGD as a Learner: From Prediction to Prescription — Closing the Loop.” *MGMT 590-037 Lecture Notes*, Summer 2026.

Submit: Push `proposal.pdf` to your team GitHub repo and share the link with `jaiswalp@purdue.edu`
 MGMT 590-037 — Project Proposal Template — Summer 2026